

Informe de Laboratorio 03

Tema: Revisión de elementos de programación parte 2

Nota

Estudiante	Escuela	Asignatura
Sebastian Arley Chirinos Negrón schirinosne@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Estructura de datos y Algoritmos Semestre: I Código: 1702124

Laboratorio	Tema	Duración
03	Revisión de elementos de programación parte 2	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 29 Mayo 2023	Al 05 Junio 2023

1. Tarea

- Lista (11 pts)
- Implementa una Lista usando POO con tipos genéricos siguiendo los estándares de Java. (Los métodos para una lista).
- Puede ignorar los siguientes métodos:
 - hashCode()
 - iterator()
 - listIterator()
 - listIterator(int index)
 - retainAll(Collection? c)
 - toArray()
 - toArray(T[] a)
- (i) Implemente una clase Node donde T es un tipo genérico, esta clase debe contener al menos dos propiedades.
- (ii) Implementar una clase List esta clase debe contener al menos esta propiedad
- public class MyList<T> extends List<T>: Esta línea declara una clase llamada MyList que extiende la clase genérica List<T>. Utiliza el parámetro de tipo genérico T para indicar que la clase trabajará con un tipo de dato específico que se determinará al usar la clase.

2. Equipos, materiales y temas utilizados

- Realizar ejercicios en temas de Estructuras de datos, tipos de datos abstractos, bucles, Arrays, Listas enlazadas, Recursión
- Java
- TAD
- Arrays
- C.m. Construye responsablemente soluciones haciendo uso de estructuras de datos y algoritmos, siguiendo un proceso adecuado para resolver problemas computacionales que se ajustan al uso de los recursos disponibles y a especificaciones concretas.

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- <https://github.com/BastleyNait/EDA-LAB-B-23A.git>
- URL para el laboratorio 04 en el Repositorio GitHub.
- <https://github.com/BastleyNait/EDA-LAB-B-23A/tree/main/lab03>

4. Ejercicios designados

4.1. Clase List

- clase abstracta genérica llamada List. Aquí hay una explicación de los elementos clave del código:

Listing 1: List.java

```
1 public abstract class List<T> {
2     private Node<T> root; // Referencia al nodo inicial de la lista
3
4     public List() {
5         this.root = null; // Inicializa la referencia al nodo inicial como null al crear la
6                             lista
7     }
8
9     public abstract int size();
10
11     public abstract boolean isEmpty();
12
13     public abstract boolean contains(Object o);
14
15     public abstract boolean add(T element);
16
17     public abstract boolean remove(Object o);
18
19     // Resto de mtodos de la lista...
20 }
```

- `Node<T> root`: Esta línea declara una variable miembro `root` que es de tipo `Node<T>`. Es una referencia al nodo inicial de la lista y almacenará el primer elemento.
- `private int size`: Esta línea declara una variable miembro `size` que representa el tamaño de la lista, es decir, el número de elementos en la lista.
- `public MyList()`: Este es el constructor de la clase `MyList`. Inicializa la referencia `root` como `null` y el tamaño de la lista como 0 al crear una nueva instancia de `MyList`.
- `public int size()`: Devuelve el tamaño de la lista.
- `public boolean isEmpty()`: Verifica si la lista está vacía.
- `public boolean contains(Object o)`: Verifica si la lista contiene un objeto específico.
- `public void printList()`: Imprime los elementos de la lista en la consola.
- `public boolean add(T data)`: Agrega un elemento al final de la lista.
- `public boolean remove(Object o)`: Elimina un objeto de la lista.
- `public T get(int index)`: Obtiene el elemento en un índice específico.
- `public void remove(int index)`: Elimina el elemento en un índice específico.

4.2. Clase MyList

- implementación concreta de la clase genérica `List`. Aquí hay una explicación de los elementos clave del código:

Listing 2: MyList.java

```
1 public class MyList<T> extends List<T> {
2     Node<T> root; // referencia al nodo inicial
3     private int size; // tamaño de la lista
4
5     // Constructor
6     public MyList() {
7         root = null;
8         size = 0;
9     }
10
11     public int size() {
12         return 0;
13     }
14
15     // Obtener la longitud de la lista
16     public int getSize() {
17         return size;
18     }
19
20     // Verificar si la lista está vacía
21     public boolean isEmpty() {
22         return size == 0;
23     }
24
25     public boolean contains(Object o) {
26         return false;
27     }
28 }
```

```
27     }
28
29     // Imprimir la lista
30     public void printList() {
31         Node<T> current = root;
32         while (current != null) {
33             System.out.print(current.getData() + " ");
34             current = current.getNextNode();
35         }
36         System.out.println();
37     }
38
39     // Aadir un elemento al final de la lista
40     public boolean add(T data) {
41         Node<T> newNode = new Node<>(data);
42         if (isEmpty()) {
43             root = newNode;
44         } else {
45             Node<T> current = root;
46             while (current.getNextNode() != null) {
47                 current = current.getNextNode();
48             }
49             current.setNextNode(newNode);
50         }
51         size++;
52         return false;
53     }
54
55     public boolean remove(Object o) {
56         return false;
57     }
58
59     // Obtener el elemento en un ndice especifico
60     public T get(int index) {
61         Node<T> current = root;
62         for (int i = 0; i < index; i++) {
63             current = current.getNextNode();
64         }
65         return current.getData();
66     }
67
68     // Eliminar el elemento en un ndice especifico
69     public void remove(int index) {
70         if (index == 0) {
71             root = root.getNextNode();
72         } else {
73             Node<T> current = root;
74             for (int i = 0; i < index - 1; i++) {
75                 current = current.getNextNode();
76             }
77             current.setNextNode(current.getNextNode().getNextNode());
78         }
79         size--;
80     }
81 }
```

- ‘ al inicio de cada punto:
- ‘public class MyList<T> extends List<T>’: Esta línea declara una clase llamada ‘MyList’ que extiende la clase genérica ‘List<T>’. Utiliza el parámetro de tipo genérico ‘T’ para indicar que la clase trabajará con un tipo de dato específico que se determinará al usar la clase.
- ‘Node<T> root’: Esta línea declara una variable miembro ‘root’ que es de tipo ‘Node<T>’. Es una referencia al nodo inicial de la lista y almacenará el primer elemento.
- ‘private int size’: Esta línea declara una variable miembro ‘size’ que representa el tamaño de la lista, es decir, el número de elementos en la lista.
- ‘public MyList()’: Este es el constructor de la clase ‘MyList’. Inicializa la referencia ‘root’ como ‘null’ y el tamaño de la lista como 0 al crear una nueva instancia de ‘MyList’.
- ‘public int size()’: Devuelve el tamaño de la lista.
- ‘public boolean isEmpty()’: Verifica si la lista está vacía.
- ‘public boolean contains(Object o)’: Verifica si la lista contiene un objeto específico.
- ‘public void printList()’: Imprime los elementos de la lista en la consola.
- ‘public boolean add(T data)’: Agrega un elemento al final de la lista.
- ‘public boolean remove(Object o)’: Elimina un objeto de la lista.
- ‘public T get(int index)’: Obtiene el elemento en un índice específico.
- ‘public void remove(int index)’: Elimina el elemento en un índice específico.

4.3. Clase Node

- ‘Node<T>’ que representa un nodo en una estructura de datos enlazada. Aquí hay una explicación de los elementos clave del código:

Listing 3: Node.java

```
1 public class Node<T> {
2     private T data; // Almacena el valor de datos del nodo
3     private Node<T> nextNode; // Referencia al siguiente nodo en la lista
4
5     // Constructor que recibe el valor de datos del nodo
6     public Node(T value) {
7         data = value;
8         this.nextNode = null; // Inicialmente,
9         // el nodo no tiene un
10        // siguiente nodo, por lo que se establece como null
11    }
12
13    // Metodo para obtener el valor de datos del nodo
14    public T getData() {
15        return data;
16    }
17
18    // Metodo para establecer el valor de datos del nodo
19    public void setData(T value) {
20        this.data = value;
```

```
21 }
22
23 // Mtodo para obtener la referencia al siguiente nodo
24 public Node<T> getNextNode() {
25     return nextNode;
26 }
27
28 // Mtodo para establecer la referencia al siguiente nodo
29 public void setNextNode(Node<T> nextNode) {
30     this.nextNode = nextNode;
31 }
32 }
```

- `private T data`: Esta línea declara una variable miembro `data` de tipo genérico `T`, que almacena el valor de datos del nodo.
- `private Node<T> nextNode`: Esta línea declara una variable miembro `nextNode` de tipo `Node<T>`, que es una referencia al siguiente nodo en la lista.
- `public Node(T value)`: Este es el constructor de la clase `Node`. Recibe un valor de datos `value` y lo asigna a la variable `data`. El siguiente nodo (`nextNode`) se establece inicialmente como `null`.
- `public T getData()`: Este método devuelve el valor de datos almacenado en el nodo.
- `public void setData(T value)`: Este método establece el valor de datos del nodo con el valor especificado.
- `public Node<T> getNextNode()`: Este método devuelve la referencia al siguiente nodo en la lista.
- `public void setNextNode(Node<T> nextNode)`: Este método establece la referencia al siguiente nodo en la lista.

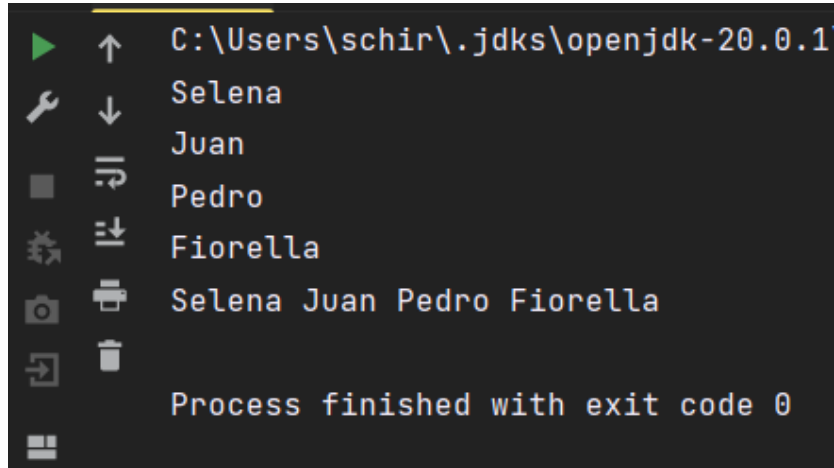
4.4. Clase test

- El programa utiliza la recursión para imprimir un triángulo de asteriscos con la cantidad de filas especificada.

Listing 4: test.java

```
1
2 public class test {
3     public static void main(String[] args) {
4         MyList<String> nombres = new MyList<String>();
5         nombres.add("Selena");
6         nombres.add("Juan");
7         nombres.add("Pedro");
8         nombres.add("Fiorella");
9         //imprimiendolo con un bucle
10        for (int i = 0; i < nombres.getSize(); i++) {
11            System.out.println(nombres.get(i));
12        }
13        //imprimeindo con el metodo implementado en MyList
14        nombres.printList();
15    }
16
17 }
```

- Esta clase es netamente para poder probar la correcta funcionalidad del la clase MyList
- Ejecucion del codigo:



```
C:\Users\schir\.jdk\openjdk-20.0.1
Selena
Juan
Pedro
Fiorella
Selena Juan Pedro Fiorella
Process finished with exit code 0
```

4.5. Commits del trabajo

- Acá estan algunos de los commits mas importantes en este trabajo:

```
commit ef762e56ad42c68a1c1ba2733a8a3fe68265ed9d
```

```
Author: BastleyNait <schirinosne@unsa.edu.pe>
```

```
Date: Mon May 22 15:52:55 2023 -0500
```

añadiendo comentarios y algunos metodos

```
commit c3147f069b5a1cacf3f7a302bbe247f3c6a6753d
```

```
Author: BastleyNait <schirinosne@unsa.edu.pe>
```

```
Date: Mon May 22 15:40:11 2023 -0500
```

getters y setters para el siguiente nodo

```
commit a92a194c2951730f266072e0e90d1265c9cffd52
```

```
Author: BastleyNait <schirinosne@unsa.edu.pe>
```

```
Date: Mon May 22 15:34:40 2023 -0500
```

añadiendo getters y setters

```
commit 5cbc190ec356aec8db1320837cfa4485765d8d8d
```

```
Author: BastleyNait <schirinosne@unsa.edu.pe>
```

```
Date: Mon May 22 15:33:43 2023 -0500
```

Creando la clase node

```
commit 46e582337f2245c5cef83ed6adb6059cc9cd9598
```

```
Author: BastleyNait <schirinosne@unsa.edu.pe>
```

```
Date: Mon May 22 10:18:17 2023 -0500
```

cambios finales lab 02

```
commit ea9c4859eed91d3dd26054f6ba4dd5b98e21fdc7
```

```
Author: Sebastian Arley <120061345+BastleyNait@users
```

```
Date: Mon May 22 10:15:12 2023 -0500
```

update informe de laboratorio

```
commit 901b602825d873cb4af5a06beb18568994a20c5f
```

```
Author: Sebastian Arley <120061345+BastleyNait@users
```

```
Date: Mon May 22 10:10:30 2023 -0500
```

añadiendo informe

```
commit b5d83ba4e88204618f48f7b49a3fd90a25eb7993
```

```
Author: BastleyNait <schirinosne@unsa.edu.pe>
```

```
Date: Mon May 15 19:07:24 2023 -0500
```

Primera version del ejercicio03

4.6. Estructura de laboratorio 03

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab03/
| .gitignore
| List.java
| MyList.java
| Node.java
| test.java
|
|
\---latex
|   EDA_lab03_schirinosne.pdf
|   EDA_lab03_schirinosne.tex
|
+---img
|   Commits.png
|   ejecucion.png
|   logo_abet.png
|   logo_episunsa.png
|   logo_unsa.jpg
|
\---src
|   List.java
|   MyList.java
|   Node.java
|   test.java
```

5. Pregunta: ¿Qué diferencia hay entre un List y un ArrayList en Java?

- La diferencia entre un List y un ArrayList en Java se encuentra en la implementación y en las características específicas de cada una:
- List es una interfaz en Java que define el comportamiento general de una lista, incluyendo operaciones como agregar elementos, acceder a elementos por índice, eliminar elementos y otras funcionalidades comunes. Es una interfaz de alto nivel que no especifica la implementación subyacente.
- ArrayList, por otro lado, es una implementación específica de la interfaz List. Representa una lista basada en un array dinámico, lo que significa que internamente utiliza un array para almacenar los elementos y se redimensiona automáticamente según sea necesario. Proporciona un acceso rápido a los elementos por índice y es eficiente para operaciones de lectura y escritura. Sin embargo, la inserción o eliminación de elementos en posiciones intermedias puede ser costosa en términos de rendimiento, ya que puede requerir desplazar los elementos.

6. Pregunta: ¿Qué beneficios y oportunidades ofrecen las clases genéricas en Java?

- En cuanto a las clases genéricas en Java, ofrecen beneficios y oportunidades significativas:

- Permiten la creación de clases y métodos que pueden ser parametrizados con tipos específicos. Esto proporciona un alto grado de reutilización de código, ya que una clase o método genérico puede ser utilizado con diferentes tipos de datos sin necesidad de escribir implementaciones específicas para cada tipo.
- Ayudan a garantizar la seguridad de tipos en tiempo de compilación. Al especificar el tipo de datos que se utilizará en una clase genérica, el compilador de Java puede detectar posibles errores de tipos antes de la ejecución del programa, lo que ayuda a evitar errores comunes y a mejorar la robustez del código.
- Mejoran la legibilidad y mantenibilidad del código. Al utilizar clases genéricas, se puede expresar de manera más clara y concisa la intención y el propósito del código. Además, al utilizar tipos genéricos, se evita la necesidad de conversiones explícitas entre tipos en muchas situaciones.
- Permiten la creación de estructuras de datos genéricas y algoritmos reutilizables. Las clases genéricas son especialmente útiles para implementar estructuras de datos como listas, pilas, colas, árboles, etc., que pueden manejar diferentes tipos de datos de manera flexible y segura.

7. Referencias

- <https://www.w3schools.com/java/>
- <https://docs.oracle.com/javase/7/docs/api/java/util/List.html>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>